

Anisa Aulia Alhaqi/18224080

function isSimetris(l: List) → **boolean**

KAMUS LOKAL

 check : **boolean**

 i : **integer**

check ← **true**

if(l.nEff > 0) **then**

while(i ≤ l.nEff/2 **and** check) **do**

if(l.contents[i] <> l.contents[l.nEff-i]) **then**

 check ← **false**

 i ← i+1

 → check

else

 → check

function plusTab(l1, l2 : List) → List

KAMUS LOKAL

 l3 : List

ALGORITMA

 i **traversal**[1..l1.nEff]

 l3 ← l1.contents[i] + l2.contents[i]

function countOccurence(l : List, x : ElType) → **integer**

KAMUS LOKAL

 count : **integer**

ALGORITMA

count ← 0

i **traversal**[1..l.nEff]

if(l.contents[i] = x) **then**

 count ← count+1

→ count

function isEqual(l1, l2 : List) → **boolean**

KAMUS LOKAL

```

        check : boolean
        i : integer
ALGORITMA
check ← true
if(l1.nEff < l2.nEff) then
    → false
else
    while(i ≤ l1.nEff and check) do
        if(l1.contents[i] < l2.contents[i] then
            check ← false
            i ← i+1
    → check

```

function index0f (l: List, x: ElType) → idxType

KAMUS LOKAL

```

        i : integer
ALGORITMA
i ← 0
while(i ≤ l.nEff and l.contents[i] < x) do
    i ← i+1
if(i = 0) then
    → idxUndef
else
    → i

```

procedure insertUnique(input/output l: List, input x: ElType)

KAMUS LOKAL

```

        i : integer
        check : boolean
ALGORITMA
i ← 0
check ← false
l.contents[l.nEff+1] ← x
while(i ≤ l.nEff and not(check)) do
    if(l.contents[i] = x) then
        check ← true

```

```

        i ← i+1
    if(check) then
        output("nilai sudah ada")
    else
        l.nEff ← l.nEff + 1

```

Jika ADT diubah menjadi implisit, yang perlu diubah adalah:
 ⇒ Tidak ada nEff pada representasi implisit

Implisit

```

function getFirstIdx(l : List) → integer
    → 0

```

```

function getLastIdx(l : List) → integer

```

KAMUS LOKAL

```

    i : integer

```

ALGORITMA

```

    i ← 0
    while(l.contents[i] <> NULL) do
        i ← i+1
    → i-1

```

Eksplisit

```

function getFirstIdx(l : List) → integer

```

KAMUS LOKAL

```

    i : integer

```

ALGORITMA

```

    i ← 1
    while(i < kapasitas and l.contents[i] = NULL) do
        i ← i+1
    → i

```

```

function getLastIdx(l : List) → integer

```

KAMUS LOKAL

```

    i : integer

```

ALGORITMA

```

    i ← kapasitas-1
    while(i ≥ 1 and l.contents[i] = NULL) do

```

```
    i ← i-1  
→ i
```

procedure closestPair(**input** l : List, **output** p1, p2 : ElType)

KAMUS LOKAL

diff : **integer**

min, p1, p2 : **integer**

ALGORITMA

min ← absolute(l.contents[1]-l.contents[2])

p1 ← l.contents[1]

p2 ← l.contents[2]

i **traversal**[2..l.nEff-1]

diff ← absolute(l.contents[i]-l.contents[i+1])

if(diff < min) **then**

min ← diff

p1 ← l.contents[i]

p2 ← l.contents[i+1]

function isFront (l1, l2 : List) → **boolean**

KAMUS LOKAL

i : **integer**

front : **boolean**

ALGORITMA

i ← 1

front ← **true**

while(i ≤ l1.nEff **and** front) **do**

if(l1.contents[i] < l2.contents[i])

front ← **false**

i ← i+1

→ front